Improved Precomputation Scheme for Scalar Multiplication on Elliptic Curves

Duc-Phong Le and Chik How Tan

Temasek Laboratories
National University of Singapore
{tslld,tsltch}@nus.edu.sg

Abstract. Precomputation is essential for window-based scalar multiplications which are the most important operation of elliptic curve cryptography. This precomputation stage may require a significant amount of time due to the expensive inversions over finite fields of large characteristic. Hence, the existing state-of-the-art precomputation schemes try to reduce the number of inversions as much as possible. However, our analysis show that the performance of precomputation schemes not only depends on the cost of field inversions, but also on the cost ratio of inversion to multiplication (i.e. I/M).

In this paper, we present a new scheme to precompute all odd multiples $[3]P, \ldots, [2k-1]P, k \geq 2$ on standard elliptic curves in affine coordinates. Our precomputation scheme strikes a balance between the number of inversions and multiplications. We show that our scheme requiring only $2(k-1)$ registers, offers the best performance in the case of $k \geq 8$ if the I/M-ratio is around 10.

Keywords: scalar multiplication, elliptic curve cryptography, precomputation scheme.

1 Introduction

The use of elliptic curve in cryptography was suggested independently by Miller [14] and Koblitz [9] in 1985. Since then, Elliptic Curve Cryptography (ECC) has received a lot of attention due to the fast group law on elliptic curves and the absence of a sub-exponential attack on the discrete logarithm problem defined over elliptic curves. Thus, it can provide the same security level as RSA but with much shorter keys.

One of the most time-consuming operation in cryptosystems based on elliptic curves is a scalar multiplication $[u]P$, where u is the scalar and P is a point on an elliptic curve over $\mathbb{F}_p$. The simplest way for scalar multiplication is *double-and-add* method that repeatedly perform doublings of a point and additions of two points from *left-to-right*. Let D denote the cost of a doubling, A denote the cost of an addition, and let $n = \lceil \log_2 u \rceil$ be the number of bits of u. The total

L. Chen (Ed.): Cryptography and Coding 2011, LNCS 7089, pp. 327–343, 2011.
© Springer-Verlag Berlin Heidelberg 2011

cost of computing $[u]P$ by the double-and-add method is approximately equal to $nD + (n/2)A$.

To reduce the number of additions from the double-and-add method, window-based algorithms, such as sliding window and 2^w-ary algorithms, were presented (see [5]). Scalar multiplication based on these methods consists of two stages: *precomputation stage* and *evaluation stage*. The former computes points $\{[3]P, [5]P, \ldots, [2k-1]P\}$, and stores these values, while the later computes scalar multiplication using these precomputed points. In this paper, we only discuss the precomputation stage.

Normally, precomputation schemes output precomputed points in affine coordinates. An evaluation stage is very efficient if a mixed addition is performed between one point in affine coordinates and another in projective coordinates (see [6] for a discussion more detail to the efficiency of ECC operations in mixed coordinates). If the point P is not fixed, the precomputation can't be performed offline and thus it requires a significant amount of time as expensive field inversions. For example, the inversion-to-multiplication (I/M) ratio is bigger than 100 on smart cards (see [18]). On workstations, for NIST-recommended elliptic curves over prime fields chosen to either be a Mersenne prime, or a Mersenne-like prime for fast modular reduction and multiplication, this ratio is roughly 80 (see benchmarks in [2]). In the cases when the Mersenne prime cannot used (e.g. pairing-based cryptography), the I/M-ratio is often reported to be 10 or higher (see benchmarks in [1]).

Due to high computation of inversion operations, many improvements have been introduced [6] [7][12][11] for the purpose of reducing the number of inversions and thus minimizing the cost of precomputation stage. All of them use the well-known trick of Montgomery which allows to compute simultaneously the inverses of n elements $a_1, a_2, \ldots, a_n$ at the cost of one inversion and $3(n-1)$ multiplications (see [4, Section 10.3.4]).

1.1 Montgomery Trick

The Montgomery trick is based on the following idea: to compute the inverse of two elements a and b, one computes their product ab and its inverse $(ab)^{-1}$. Then, the inverse of a and b can be computed by $a^{-1} = b \cdot (ab)^{-1}$ and $b^{-1} = a \cdot (ab)^{-1}$.

For n elements $a_1, a_2, \ldots, a_n$ of a finite field $\mathbb{F}_p$, Montgomery trick computes their inverses $b_1, b_2, \ldots, b_n$ as follows:

Algorithm 1. Montgomery Trick for computing simultaneously n inverses

Input: $a_1, a_2, \ldots, a_n$;
Output: The inverses b_i of a_i, for any $i \in [1, \ldots, n]$;

$c_1 \leftarrow a_1$
for $i = 2$ **to** n **do**
  $c_i \leftarrow c_{i-1} \cdot a_i$
end
$u \leftarrow c_n^{-1}$
for $i = n$ **down to** 2 **do**
  $b_i \leftarrow c_{i-1} \cdot u$
  $u \leftarrow ua_i$
end
$b_1 \leftarrow u$

The cost of n inversions using this trick is $1\mathsf{I} + 3(n-1)\mathsf{M}$. For n large, it can asymptotically achieve an I/M-ratio of 3. However to apply this trick, all values to invert must be known. The precomputation in (multi-)scalar multiplication does not satisfy this, since each point depends on a previous computed point, *e.g.* $[3]P = [2]P + P$, $[5]P = [3]P + [2]P$.

1.2 Previous Work

Since there exists a dependence between points needed to precompute as mentioned above, the naive method first precomputes $[2]P$ in affine coordinates, then $[x]P$ in affine coordinates for $3 \leq x \leq (2k-1)$, and x odd. This approach requires k field inversions.

There were several approaches to overcome this dependence. Cohen et al. [6] divide the set of points needed to precompute into $(\lceil \log_2 k \rceil + 1)$ subgroups of *independent* points. The ith group, for $i \in [1, \lceil \log_2 k \rceil]$ consists of $[2^i]P$ and the odd multiples $[2^{i-1} + j]P$ for odd $j \in [1, (2^{i-1} - 1)]$. The inversions required in each step are computed simultaneously using the Montgomery trick. This reduces the number of inverses from k to $(\lceil \log_2 k \rceil + 1)$.[1]

In another approach, by generalizing the method of computing $[3]P$ in [3], Dahmen et al. [7] presented recursive formulas to express all odd points $[3]P, [5]P, \ldots, [2k-1]P$ with only known parameters. Then, all values are computed by only one single field inversion due to the Montgomery trick. Their scheme requires only one inversion but adds an enormous number of multiplications and squarings (see Table 1).

Another way to obtain precomputed points with just one inversion can be executed as follows: one first computes the points separately in projective coordinates $\mathcal{P}$ (or Jacobian coordinates $\mathcal{J}$) that allow us to avoid the expensive field inversion and then convert these points to affine coordinates $\mathcal{A}$ at the end. In curves of Weierstrass form, a conversion from $\mathcal{P}$ to $\mathcal{A}$ needs $2\mathsf{M} + \mathsf{I}$ and from Jacobian coordinates $\mathcal{J}$ to $\mathcal{A}$ needs $3\mathsf{M} + \mathsf{S} + \mathsf{I}$. The inversions required for the

[1] If we divide the n elements into m subgroups and apply Montgomery trick for each subgroup then the cost is $mI + 3(n-m)\mathsf{M}$ (see [15, Section 4.1]).

conversion are computed simultaneously using the Montgomery trick. In that direction, Longa and Miri [12] presented an efficient precomputation scheme using co-Z addition formula [13]. Their scheme requires $(k-2)$ multiplications and $(k-2)$ squarings less than the scheme in [7]. On the other hand, the Longa-Miri scheme requires at least $(k+2)$ registers[2] more than that in [7] (see Table 1).

Recently, Okeya, Takagi and Vuillaume [16] introduced a very elegant observation which allow to save one inversion when computing $P \pm Q$ in affine coordinates. In an invited talk at ECC 2008, Scott suggested that this observation can be used in precomputation schemes [17]. Longa and Gebotys [11] then expanded this idea and presented novel precomputation schemes for different curves: Weierstrass, Jacobi quartic and Edwards curves.

Consider the method of Cohen et al. [6] that did not benefit from the Okeya et al. observation, we call the point $[2^i]P$ the *strategic point* in the ith step, with $i \leq \lceil \log_2 k \rceil$. In the next step, one computes the points $[2^i+j]P$, with $j \in [1, 2^i-1]$ and the next strategic point $[2^{i+1}]P$. Points $[2^i+j]P$ are computed from the last strategic point $[2^i]P$. The *first strategic point* in Cohen et al.'s scheme is $[2]P$. Like the scheme of Cohen et al. [6], Longa-Gebotys's scheme divides the set of points needed to precompute into subgroups and makes a doubling of the strategic point for the next step. However their strategy is slightly different from that in [6], the Longa-Gebotys's scheme begins with the *first strategic point* of $[3]P$, and it thus benefits from the Okeya et al.'s observation (see [11] for more details).

The complexity of precomputation schemes discussed above are showed in the following table, where I, S and M denote an inversion, a squaring and a multiplication, respectively.

Table 1. Comparative efficiency of some precomputation algorithms

Scheme	**Cost**	**Number of registers**
Dahmen et al. [7]	$1\mathsf{I} + (10k-11)\mathsf{M} + (4k)\mathsf{S}$	$2(k-1)$ registers
Longa-Miri [12]	$1\mathsf{I} + 9(k-1)\mathsf{M} + (3k+2)\mathsf{S}$	$3k$ registers
	$1\mathsf{I} + 9(k-1)\mathsf{M} + (2k+4)\mathsf{S}$	$(4k-3)$ registers
Longa-Gebotys [11] (Jacobi Quartics)	$1\mathsf{I} + (12k-16)\mathsf{M} + (5k+2)\mathsf{S}$	$5k$ registers
Longa-Gebotys [11] (Edwards curves)	$1\mathsf{I} + (15k + \lceil \frac{k-3}{k-1} \rceil - 14)\mathsf{M} + (k+2)\mathsf{S}$	$3k$ registers
Cohen et al. [6]	$(\lceil \log_2 k \rceil + 1)\mathsf{I} + (5k + 2\lceil \log_2 k \rceil - 8)\mathsf{M} + (k + 2\lceil \log_2 k \rceil - 1)\mathsf{S}$ (requires $2(k-1)$ registers)	

It is clear that the Longa-Miri scheme [12] is better than that of Dahmen et al. [7]. In comparison to Cohen et al. scheme [6], [12] requires only one inversion while [6] needs $(\lceil \log_2 k \rceil + 1)$ inversions. However, [6] requires much fewer multiplications and squarings than [12] (see Table 1).

[2] Storing a point on an elliptic curve requires 2 registers for affine coordinates and 3 registers for projective coordinates.

Our following simple analysis shows that the performance of precomputation schemes depend not only on the cost of field inversions, but also on the cost ratio of inversion to multiplication and the width of the window used by window-based scalar-multiplication algorithms.

We assume here that the cost ratio of squaring to multiplication (S/M) is equal to 0.8. The comparison depends on the window width k.

- $\lceil \log_2 k \rceil = 3$: the Longa-Miri scheme is better than Cohen et al.'s scheme if $\mathsf{I} > 10.2\mathsf{M}$.
- $\lceil \log_2 k \rceil = 4$: the Longa-Miri scheme is better than Cohen et al.'s scheme if $\mathsf{I} > 13.8\mathsf{M}$.
- $\lceil \log_2 k \rceil = 5$: the Longa-Miri scheme is better than Cohen et al.'s scheme if $\mathsf{I} > 27.7\mathsf{M}$.

If the I/M ratio is around 13, which is realistic in recent software implementations on workstations (see benchmarks in [1]), then the scheme of Cohen et al. is better than Longa-Miri scheme with $\lceil \log_2 k \rceil \geq 4$. Moreover, we should recall that the Longa-Miri scheme makes use of $(4k-3)$ registers, therefore requires twice as many registers in comparison to the Cohen et al.'s scheme.

1.3 Contributions

Motivated by the above simple analysis, we investigate on a new precomputation strategy and propose a new precomputation scheme. Our method is based on that in [6][11], but our scheme begins with the first strategic point of $4P$, and we perform a tripling of the strategic point in each step.

Our contribution is as follows:

- On curves of Weierstrass form, we propose a new precomputation scheme which directly compute points in affine coordinates. Our scheme requires no more than $(\lceil \log_3 \frac{k}{2} \rceil + 2)$ inversions. Compared to schemes in the same approach [6][11], our scheme needs fewer inversions. We also show that our scheme is better than [12] if the I/M-ratio is not very large.
- Furthermore, our scheme requires only $2(k-1)$ registers. In comparison to the Longa-Miri scheme which is very efficient if the I/M-ratio is large, our scheme needs less memory. We save at least $k+2$ registers (see Table 1).
- We analyze in detail the computational complexity of schemes and make a comparison between schemes, point out which schemes obtain the highest performance with a particular range of I/M and particular width of the window.

The rest of the paper is organized as follows: Section 2 briefly recall some definitions on Elliptic Curve Cryptography. Some essential observations to improve precomputation schemes are presented in Section 3. Section 4 present a new precomputation scheme which is directly computed in affine coordinates on a standard elliptic curve. We make a comparison in Section 5 and a conclusion in Section 6.

2 Elliptic Curve Cryptography

In this section, we briefly recall some basic definitions in Elliptic Curve Cryptography (for the background, we refer to [8]).

2.1 Elliptic Curves over Finite Fields

For p prime and $p > 3$, an elliptic curve defined over $\mathbb{F}_p$ in short Weierstrass form is the set of solutions (x, y) to the following equation:

$$E : y^2 = x^3 + ax + b,$$

together with an extra point $\mathcal{O}$ which is called the point at infinity. Where $a, b \in \mathbb{F}_p$ such that the discriminant $\Delta = -16(4a^3 + 27b^2)$ is non-zero.

We usually use the notation $E(\mathbb{F}_p)$ for the set of points (x, y) on E with affine coordinates in the field $\mathbb{F}_p$ together with the point $\mathcal{O}$, the identity element of the group. Points on an elliptic curve can be represented in several coordinate systems, such as affine ($\mathcal{A}$), projective ($\mathcal{P}$), Jacobian ($\mathcal{J}$) coordinates.

The set of points on an elliptic curve forms a group under a certain addition rule. Let $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ be two points on elliptic curve E with $P_1 \neq -P_2$. Then the coordinates of $P_3 = P_1 + P_2 = (x_3, y_3)$ can be computed as follows:

$$x_3 = \lambda^2 - x_1 - x_2, \qquad y_3 = \lambda(x_1 - x_3) - y_1, \quad \text{where}$$

$$\lambda = \frac{3x_1^2 + a}{2y_1} \text{ if } P_1 = P_2, \quad \lambda = \frac{y_2 - y_1}{x_2 - x_1} \text{ if } P_1 \neq P_2.$$

The doubling and addition require $1\mathsf{I} + 2\mathsf{M} + 2\mathsf{S}$ and $1\mathsf{I} + 2\mathsf{M} + 1\mathsf{S}$ respectively in affine coordinates, where I, M and S denote field inversion, field multiplication and field squaring, respectively.

Projective coordinates. Since inversion in $\mathbb{F}_p$ is significantly more expensive than multiplication, it is advantageous to represent points using projective coordinates of which several types have been proposed.

In *standard projective coordinates* abbreviated as $\mathcal{P}$, a point $P = (x, y)$ in affine coordinates is represented by the triplet (X, Y, Z), where $(X, Y, Z) = (\lambda X, \lambda Y, \lambda Z)$ for all nonzero λ. The elliptic curve equation for these coordinates is

$$Y^2 Z = X^3 + aXZ^2 + bZ^3.$$

In projective coordinates, the negative of a point $P = (X_1, Y_1, Z_1)$ is given by $-P = (X_1, -Y_1, Z_1)$.

In *Jacobian coordinates* abbreviated as $\mathcal{J}$, a point $P = (x, y)$ in affine coordinates is represented by the triplet (X, Y, Z), where $(X, Y, Z) = (\lambda^2 X, \lambda^3 Y, \lambda Z)$ for all nonzero λ. The elliptic curve equation becomes

$$Y^2 = X^3 + aXZ^4 + bZ^6.$$

2.2 Scalar Multiplication

A *scalar multiplication* $[u]P$ of a point $P \in E(\mathbb{F}_p)$ and a scalar $u > 0$ is defined by adding P to itself u times.

$$\begin{aligned}[u] \ &: E \to E \\ P &\mapsto [u]P = \underbrace{P + P + \cdots + P}_{\text{u times}}.\end{aligned}$$

Scalar multiplication is the core operation in many elliptic curve cryptographic protocols. The standard method to compute a scalar multiplication is the *double-and-add* algorithm, which requires n doublings and on average $\frac{n}{2}$ additions for integers of length $n = \log_2 u$.

Many improvements introduced tend to decrease the number of additions, leading to a considerable speedup. Almost all algorithms require precomputed tables.

3 Preliminary Observations

Let $P \in E(\mathbb{F}_p)$ be a point needed a scalar multiplication. Some observations that can be used to speedup precomputations in affine coordinates are noted in this section.

Okeya-Takagi-Vuillaume's observation. Okeya et al. pointed out in [16] that one can save one field inversion when computing $P_1 \pm P_2$ in affine coordinates $\mathcal{A}$. That is because two additions $P_1 + P_2$ and $P_1 - P_2$ make use of a common inversion $1/(x_{P_1} - x_{P_2})$ due to the same x-coordinate of both points P_2 and $-P_2$ on elliptic curves.

Longa-Gebotys's precomputations. Longa and Gebotys [11] benefited from this observation for precomputation scheme. Their scheme thus saves some inversions in the finite field in comparison to straightforward scheme. They call $P_1 - P_2$ the conjugate addition to $P_1 + P_2$. They choose $[3]P$ as the first strategic point and double this point for next step (see [11]).

Novel strategy. We observe that the next strategic point computed by doubling the strategic point being computed is not optimized to benefit from the Okeya-Takagi-Vuillaume observation. We show that the next strategic point should be computed by doubling the point being computed and then adding $[2]P$, so that computations of the next step can cover all odd points from the point being computed to the point being sum of the point being computed and the next base point. Our strategy profit all odd points precomputed, and collect a maximum number of points into the subgroup being computed.

Definition 1. *Define*

$$
\begin{aligned}
&1. \quad u_i = \begin{cases} 1 & \textit{if} \quad i = 0, \\ n_{i-1} + u_{i-1}, & \textit{if} \quad i \geq 1, \\ 2k-1, & \textit{if} \quad n_{i-1} + u_{i-1} \geq 2k-1. \end{cases} \\
&2. \quad n_i = \begin{cases} 2u_i + 2, & \textit{if} \quad u_i < k-1, \\ 2n_{i-1}, & \textit{if} \quad u_i \geq k-1 \geq n_{i-1}, \\ n_{i-1}, & \textit{if} \quad n_{i-1} \geq k. \end{cases}
\end{aligned} \tag{1}
$$

In the following section, we will present a new precomputation scheme such that u_i becomes the coefficient of the *point being computed* and n_i becomes the coefficient of the *strategic point* at step i. We first choose, $u_0 = 1$, and $n_0 = 4$, and thus the first strategic point in our scheme will be $[4]P$.

Lemma 1. *Let u_i, n_i be defined as in the Definition 1. For $u_i < k-1$, we have:*

$$n_{i+1} = 3n_i. \tag{2}$$

Proof. By definition, we have:

$$
\begin{aligned}
n_i &= 2u_i + 2, \\
u_{i+1} &= n_i + u_i,
\end{aligned}
$$

Then, the coefficient of the next strategic point n_{i+1} must be equal to two times of the coefficient of the point being computed u_{i+1} plus 2, so:

$$n_{i+1} = 2u_{i+1} + 2 = 2n_i + 2u_i + 2 = 3n_i.$$

The lemma is proven. □

By Definition 1, the coefficient of next strategic point n_{i+1} is computed by doubling the coefficient of the point being computed u_{i+1} plus 2. Updating the next strategic point thus requires one doubling and one addition ($[n_{i+1}]P = [2u_i]P + [2]P$) which cost $2\mathsf{I}+3\mathsf{S}+4\mathsf{M}$. By using Lemma 1, we can update the next strategic point by tripling the strategic point being computed ($[n_{i+1}]P = [3n_i]P$) that costs $1\mathsf{I} + 4\mathsf{S} + 7\mathsf{M}$ (see [3]).

Example 1. We give here scenarios for cases $k = 4, 8, 16$.

- When $k = 4$, one first computes $\{[4]P\}$, then $\{[4 \pm 1]P\}$, and finally $\{[7]P\}$ $(= [4]P + [3]P)$.
- When $k = 8$, one first computes $\{[4]P\}$, then $\{[4 \pm 1]P, [12]P\}$, and finally $\{[12 \pm 1]P, [12 \pm 3]P, [12 - 5]P\}$.
- When $k = 16$, one first computes $\{[4]P\}$, then $\{[4 \pm 1]P, [12]P\}$, and then $\{[12 \pm 1]P, [12 \pm 3]P, [12 \pm 5]P, [24]P\}$, and finally $\{[24 \pm 1]P, [24 \pm 3]P, [24 \pm 5]P, [24 + 7]P\}$.

Using these observations, the following section describes a new precomputation scheme in affine coordinates.

4 Precomputation Scheme in Affine Coordinates

4.1 Algorithm

In principle, our scheme is similar to that in [6][11], but differs in the strategy to choose the next base point. Due to this strategy, we can collect more points in each subgroup than in [6][11], thus benefit more from conjugate addition and need fewer inversions.

Our precomputation scheme works as follows.

Algorithm 2. Precomputation Scheme in Affine Coordinates

Input: k, $P \in E[r]$.
Output: Points $[i]P$ for any $i \in \{3, \ldots, (2k-1)\}$.

Step 1. 1.1. Compute $[4]P$ by using the algorithm [10]
1.2. $n_0 \leftarrow 4,\ u_0 \leftarrow 1,\ i \leftarrow 0$

Step 2.
repeat
2.1. $d \leftarrow 1,\quad u_{i+1} \leftarrow \min\{u_i + n_i, 2k-1\},\quad n_{i+1} \leftarrow n_i$
if $k > u_{i+1} + 1$ **then**
`// In the case the next potential strategic point` $T = [3n_i]P$ `is less than` $[2k]P$.
2.2. $d \leftarrow$ denominator needing for tripling $[n_i]P$
2.3. $n_{i+1} \leftarrow 3n_i$
else if $(n_i < k \leq u_{i+1} + 1)$ **then**
`// double the base point,` $T = [2n_i]P$
2.4. $d \leftarrow 2y_{[n_i]P},\quad n_{i+1} \leftarrow 2n_i$

2.5. Compute simultaneously inverses of d and $(x_{[n_i]P} - x_{[j]P})$ for all odd numbers $j \in [1, u_i]$
2.6. Compute points T and $[n_i \pm j]P$
2.7. $i \leftarrow i + 1$
until $(u_i + n_{i-1} > 2k - 1)$
Step 3.
if $u_i < 2k - 1$ **then**
`// using` *`Montgomery trick`* `and` $[j]P$ `have been computed. For all odd numbers` $j \in [u_{i-1} + 2, 2k - n_{i-1} - 1]$
3.1. Compute simultaneously inverses of $(x_{[n_{i-1}]P} - x_{[j]P})$
3.2. Compute points $\{[n_{i-1} + j]P\}$ using $(x_{[n_{i-1}]P} - x_{[j]P})^{-1}$
return

Remark 1. In our algorithm, we make use of the direct quadruple $[4]P$ in [10] which costs $1\mathsf{I} + 7\mathsf{S} + 8\mathsf{M}$.

Remark 2. The computations in Algorithm 2 requires only $2(k-1)$ registers.

Theorem 1 (Completeness of the algorithm). *Upon terminating, the scheme outputs all odd points* $\{[3]P, [5]P, \ldots, [2k-1]P\}$.

Proof. Let l be the number of triplings performed in Step 2. Note that, $n_l = 4 \cdot 3^l$. Since, $n_l < 2k < 3n_l$, it follows that

$$4 \cdot 3^l < 2k < 4 \cdot 3^{l+1}, \text{ or } \log_3(k/2) - 1 < l < \log_3(k/2).$$

Define

$$l = \lceil \log_3(k/2) \rceil - 1, \text{ or } l = \lfloor \log_3(k/2) \rfloor. \tag{3}$$

Step 1 of Algorithm 2 computes $[4]P$. The coefficient of the point being computed is now $u_i = 1$ and the coefficient of the strategic point is $n_i = 4$.

Step 2, computes all odd points from $[u_i + 2]P$, to $\min\{[n_i + u_i]P, [2k-1]P\}$. That is :

$$n_i - u_i = 2u_i + 2 - u_i = u_i + 2.$$

So, Step 2 can compute odd points from $[u_i + 2]P$ to $[n_i + u_i]P$ or $[2k-1]P$. Algorithm 2 terminates in the following cases :

1. Case 1: if $u_{l+1} \geq 2k-1$. The algorithm outputs all odd points $[3]P, \ldots, [2k-1]P$ and terminates in Step 2. For example, when $k = 8$, $l = 1$ $(= \lfloor \log_3(4) \rfloor)$, $u_1 = 5$, $n_1 = 12$. (see Example 1, §3).
2. Case 2: if $u_{l+1} < 2k-1$ and $n_l \geq k$. The algorithm terminates in Step 3. This is the case of $k = 4$ (see Example 1, §3).
3. Case 3: if $n_l < k \leq u_{l+1}$. In this case, the algorithm will double the strategic point n_l and terminates in Step 2. For example, when $k = 16$, $l = 1$ $(= \lfloor \log_3(8) \rfloor)$, $n_1 (= 12) < k < u_2 (= 17)$. (see Example 1, §3).
4. k can't be greater than u_{l+1}. Because $u_{l+1} = n_l + u_l = \frac{3}{2} n_l - 1$ (as defined in Definition 1), and $2k < 3n_l$ (as assumption).

When Case 2 happens, Algorithm 2 terminates in Step 3. Because:

$$u_{l+1} + n_l = u_l + 2n_l > 2k - 1, \text{ (as } n_l \geq k).$$

When Case 3 happens, Algorithm 2 terminates in Step 2 as :

$$n_{l+1} + u_{l+1} = 2n_l + n_l + u_l > 3n_l > 2k.$$

The theorem is proven. □

Theorem 2. *Let* Case i, *where* $i = \{1, 2, 3\}$ *be cases in which Algorithm 2 terminates as indicated in the above theorem. In terms of affine coordinates, our precomputation scheme which computes the set of points* $\{[3]P, [5]P, \ldots, [2k-1]P\}$ *costs*

(i) $(l+2)\mathsf{I} + (k + 4l + 6)\mathsf{S} + (2k + \frac{1}{2}3^{l+2} + 7l + 3/2)\mathsf{M}$ *in the* Case 1,
(ii) $(l+3)I + (k + 4l + 6)\mathsf{S} + (5k - 3^{l+2}/2 + 7l - 3/2)\mathsf{M}$ *in* Case 2,
(iii) $(l+3)\mathsf{I} + (k + 4l + 8)\mathsf{S} + (2k + 3^{l+2}/2 + 7l + 3m + 7/2)\mathsf{M}$ *in the* Case 3,

where $l = \lceil \log_3(k/2) \rceil - 1$ *and* $m = \max\{3^l, k - 4 \cdot 3^l\}$.

Proof. In Step 1 of Algorithm 2, we need $1\mathsf{I} + 7\mathsf{S} + 8\mathsf{M}$ for computing $4P$ [10].

$$C_1 = 1\mathsf{I} + 7\mathsf{S} + 8\mathsf{M}. \tag{4}$$

In Step 2, the following computations are performed l times. Firstly we perform 3 squarings and 2 multiplications for the computation of d (step 2.1). Secondly we perform $(u_i + 1)/2 + 1$ inversions using Montgomery trick that requires one inversion and $3/2(u_i + 1)$ multiplications (step 2.4). Thirdly, we triple the point $[n_i]P$ and compute points $[n_i \pm j]P$ (step 2.5). We need one squaring plus 5 multiplications for tripling $[n_i]P$ and one squaring plus two multiplications for each addition (we have $(u_i + 1)$ additions). Thus, for each time, we perform one inversion, $7/2(u_i + 3)$ multiplications and $(u_i + 5)$ squarings. We have,

$$n_i = 4 \cdot 3^i. \tag{5}$$

$$\begin{aligned} u_i = u_{i-1} + n_{i-1} &= u_{i-1} + 4 \cdot 3^{i-1} \\ &= u_{i-2} + 4 \cdot 3^{i-2} + 4 \cdot 3^{i-1} \\ &= 1 + 4\sum_{j=0}^{i-1} 3^j = 1 + 2(3^i - 1) \\ &= 2 \cdot 3^i - 1, \end{aligned} \tag{6}$$

and

$$\sum_{i=0}^{l-1} u_i = \sum_{i=0}^{l-1} (2 \cdot 3^i - 1) = 3^l - l - 1. \tag{7}$$

In total, we perform :

$$\begin{aligned} C_2 &= \sum_{i=0}^{l-1} (1\mathsf{I} + (u_i + 5)\mathsf{S} + 7/2(u_i + 3)\mathsf{M} \\ &= l \cdot \mathsf{I} + (3^l + 4l - 1)\mathsf{S} + 7/2(3^l + 2l - 1)\mathsf{M}. \end{aligned} \tag{8}$$

The cost of Step 2 without tripling is :

$$\begin{aligned} C_2^{(1)} &= 1\mathsf{I} + (u_i + 5)\mathsf{S} + 7/2(u_i + 3)\mathsf{M} - (4\mathsf{S} + 10\mathsf{M}) \\ &= 1\mathsf{I} + (u_i + 1)\mathsf{S} + 1/2(7u_i + 1)\mathsf{M} \\ &= 1\mathsf{I} + 2 \cdot 3^i\mathsf{S} + (7 \cdot 3^i - 3)\mathsf{M}. \end{aligned} \tag{9}$$

The cost of Step 2 without tripling but with doubling $[n_i]P$ is :

$$\begin{aligned} C_2^{(2)} &= 1\mathsf{I} + (u_i + 5)\mathsf{S} + 7/2(u_i + 3)\mathsf{M} - (2\mathsf{S} + 5\mathsf{M}) \\ &= 1\mathsf{I} + (u_i + 3)\mathsf{S} + 1/2(7u_i + 11)\mathsf{M} \\ &= 1\mathsf{I} + (2 \cdot 3^i + 2)\mathsf{S} + (7 \cdot 3^i + 2)\mathsf{M}. \end{aligned} \tag{10}$$

Algorithm 2 terminates in one of three following cases:

- Case 1: if $n_l + u_l \geq 2k - 1$. The algorithm terminates in Step 2. In this case, after performing l times Step 2 their cost is C_2. $(2k-1-u_l)/2$ points are still needed to be computed (so, $(2k-1-u_l)/2$ additions). We have to perform $(n_l - u_l - 1)/2$ inversions. Thus, the cost is :

$$\begin{aligned} C_2^{(3)} &= (2k - u_l - 1)/2(1\mathsf{S} + 2\mathsf{M}) + 1\mathsf{I} + 3((n_l - u_l - 1)/2 - 1)\mathsf{M} \\ &= 1\mathsf{I} + (2k - u_l - 1)\mathsf{S}/2 + (4k + 3n_l - 5u_l - 11)\mathsf{M}/2 \\ &= 1\mathsf{I} + (k - 3^l)\mathsf{S} + (2k + 3^l - 3)\mathsf{M}. \end{aligned} \tag{11}$$

  In total, the cost in the Case 1 is:

$$\begin{aligned} C &= C_1 + C_2 + C_2^{(3)} \\ &= 1\mathsf{I} + 7\mathsf{S} + 8\mathsf{M} + l \cdot I + (3^l + 4l - 1)\mathsf{S} + 7/2(3^l + 2l - 1)\mathsf{M} \\ &\qquad + 1\mathsf{I} + (k - 3^l)\mathsf{S} + (2k + 3^l - 3)\mathsf{M} \\ &= (l + 2)\mathsf{I} + (k + 4l + 6)\mathsf{S} + (2k + \frac{1}{2}3^{l+2} + 7l + 3/2)\mathsf{M}. \end{aligned} \tag{12}$$

  In this case, it is easy to see that the number of conjugate additions computed is equal to $k - \frac{n_l}{2} - \frac{u_l - 1}{4} = k - \frac{3^{l+1}+1}{2}$.

- Case 2: if $n_l > k$ and $2k - 1 > n_l + u_l$. The algorithm terminates in Step 3. In this case, we firstly perform l times Step 2 with the cost C_2. Then, we perform Step 2 without tripling, that costs $C_2^{(1)}$. Finally, we perform $(2k-1-u_{l+1})/2$ additions for computing points from $[u_{l+1} + 2]P$ to $[2k - 1]P$. The cost of the last step is:

$$\begin{aligned} C_3 &= (2k - 1 - u_{l+1})/2(1\mathsf{S} + 2\mathsf{M}) + 1\mathsf{I} + 3((2k - 1 - u_{l+1})/2 - 1)\mathsf{M} \\ &= 1\mathsf{I} + (k - 3^{l+1})\mathsf{S} + (5k - 5 \cdot 3^{l+1} - 3)\mathsf{M}. \end{aligned} \tag{13}$$

  In total, the cost in the Case 2 is:

$$\begin{aligned} C &= C_1 + C_2 + C_2^{(1)} + C_3 \\ &= 1\mathsf{I} + 7\mathsf{S} + 8\mathsf{M} + l \cdot I + (3^l + 4l - 1)\mathsf{S} + 7/2(3^l + 2l - 1)\mathsf{M} \\ &\quad + 1\mathsf{I} + 2 \cdot 3^l\mathsf{S} + (7 \cdot 3^l - 3)\mathsf{M} + 1\mathsf{I} + (k - 3^{l+1})\mathsf{S} + (5k - 5 \cdot 3^{l+1} - 3)\mathsf{M} \\ &= (l + 3)I + (k + 4l + 6)\mathsf{S} + (5k - 3^{l+2}/2 + 7l - 3/2)\mathsf{M}. \end{aligned} \tag{14}$$

  In Case 2, a short calculation shows that the number of conjugate additions computed is equal to $\frac{3^{l+1}-1}{2}$.

– Case 3: if $u_{l+1} \geq k > n_l$. The algorithm also terminates in Step 2. In this case, the algorithm firstly perform l times Step 2 with the cost C_2. Then, Algorithm perform Step 2 without tripling but with doubling $[n_i]P$, that costs $C_2^{(2)}$. Finally, we perform $(2k-1-u_{l+1})/2$ additions for computing points from $[u_{l+1}+2]P$ to $[2k-1]P$. We have, $n_{l+1} = 2n_l$ and $u_{l+1} = u_l + n_l$. Let $m = \max\{(n_{l+1}-1-u_{l+1})/2, (2k-n_{l+1})/2\} = \max\{3^l, k-4\cdot 3^l\}$ be the number of inversions in this step that cost $1\mathsf{I} + 3(m-1)\mathsf{M}$ by using Montgomery trick. The cost of the last step is:

$$\begin{aligned} C_2^{(4)} &= (2k-1-u_{l+1})/2(1S+2M) + 1\mathsf{I} + 3(m-1)\mathsf{M} \\ &= 1\mathsf{I} + (k-3^{l+1})\mathsf{S} + (2k-2\cdot 3^{l+1}+3m-3)\mathsf{M}. \end{aligned} \tag{15}$$

In total, the cost in the Case 3 is:

$$\begin{aligned} C &= C_1 + C_2 + C_2^{(2)} + C_2^{(4)} \\ &= 1\mathsf{I} + 7\mathsf{S} + 8\mathsf{M} + l\cdot\mathsf{I} + (3^l+4l-1)\mathsf{S} + 7/2(3^l+2l-1)\mathsf{M} + 1\mathsf{I} \\ &+ 2(3^l+1)\mathsf{S} + (7\cdot 3^l+2)\mathsf{M} + 1\mathsf{I} + (k-3^{l+1})\mathsf{S} + (2k-2\cdot 3^{l+1}+3m-3)\mathsf{M} \\ &= (l+3)\mathsf{I} + (k+4l+8)\mathsf{S} + (2k+3^{l+2}/2+7l+3m+7/2)\mathsf{M}. \end{aligned} \tag{16}$$

In Case 3, the number of conjugate additions computed is equal to $\frac{3^{l+1}-1}{2} + m'$, where $m' = \min\{3^l, k-4\cdot 3^l\}$.

We summarize the cost of each step of Algorithm 2 in Table 2. □

Table 2. Cost of precomputation stage

Step 1	$1\mathsf{I} + 7\mathsf{S} + 8\mathsf{M}$
Step 2 ($\times l$ times)	$l\cdot\mathsf{I} + (3^l+4l-1)\mathsf{S} + 7/2(3^l+2l-1)\mathsf{M}$
Step 2 (without tripling)	$1\mathsf{I} + 2\cdot 3^i\mathsf{S} + (7\cdot 3^i-3)\mathsf{M}$
Step 2 (without tripling, but doubling)	$1\mathsf{I} + (2\cdot 3^i+2)\mathsf{S} + (7\cdot 3^i+2)\mathsf{M}$
Step 3	$1\mathsf{I} + (k-3^{l+1})\mathsf{S} + (5k-5\cdot 3^{l+1}-3)\mathsf{M}$
Case 1	$(l+2)\mathsf{I} + (k+4l+6)\mathsf{S} + (2k+\frac{1}{2}3^{l+2}+7l+3/2)\mathsf{M}$
Case 2	$(l+3)I + (k+4l+6)\mathsf{S} + (5k-3^{l+2}/2+7l-3/2)\mathsf{M}$
Case 3	$(l+3)\mathsf{I} + (k+4l+8)\mathsf{S} + (2k+3^{l+2}/2+7l+3m+7/2)\mathsf{M}$

5 Performance Comparison

In this section, we compare the proposed scheme in Section 4 with the existing schemes on standard elliptic curves in the literature.

The advantage of a method depends on I/M ratios and S/M-ratios over prime fields. In this analysis, the ratio of a field squaring to a field multiplication is set to be $\mathsf{S} = 0.8\mathsf{M}$ as commonly used in the literature, see [2]. We should point out that our algorithm requires fewer inversions in comparison to straightforward algorithm and other algorithms in [6][11] (see Table 3), but requires more inversions than algorithms in [7][12] (see Table 4). Therefore, the performance of the new algorithm depends on the ratio between one inversion and one multiplication.

In general, one inversion is much more expensive than one multiplication. The I/M-ratios deeply depend on many factors such as the implementations, hardware architecture, the prime characteristic of finite fields, the size of finite fields, etc [1]. On smart cards equipped with a cryptographic co-processor, I/M ratio is bigger than 100 [18] (see §1). For software implementations on workstations of an inversion in a prime field, as pointed out in [1], the I/M ratios can be estimated to be roughly 80 for NIST-recommended elliptic curves (benchmarks in [2]) and to be around 10 or higher for curves over general prime finite field.

In the following comparisons, we don't fix the I/M-ratio, we only show the complexity of each scheme and point out which scheme obtains the highest performance with a particular range of I/M and particular width of the window.

Table 3 gives a comparison between our scheme with schemes in [6][11] which have the same approach with ours.

Table 3. Compare the performance of precomputation schemes on Weierstrass curves in the same approach

k	Cohen et al. [6]	Longa-Gebotys [11]	Our algorithm
4	$3\mathsf{I} + 16\mathsf{M} + 7\mathsf{S}$	$3\mathsf{I} + 13\mathsf{M} + 8\mathsf{S}$	$3\mathsf{I} + 13\mathsf{M} + 8\mathsf{S}$[3]
8	$4\mathsf{I} + 38\mathsf{M} + 13\mathsf{S}$	$6\mathsf{I} + 23\mathsf{M} + 14\mathsf{S}$	$3\mathsf{I} + 38\mathsf{M} + 18\mathsf{S}$
16	$5\mathsf{I} + 80\mathsf{M} + 23\mathsf{S}$	$11\mathsf{I} + 41\mathsf{M} + 24\mathsf{S}$	$4\mathsf{I} + 68\mathsf{M} + 28\mathsf{S}$
32	$6\mathsf{I} + 162\mathsf{M} + 41\mathsf{S}$	$22\mathsf{I} + 79\mathsf{M} + 44\mathsf{S}$	$5\mathsf{I} + 132\mathsf{M} + 46\mathsf{S}$[4]

From Table 3, it is clearly that our scheme requires the least number of inversions. It offers the best performance if the cost of one inversion is more expensive than the cost of 7 multiplications for all cases $k = 4, 8, 16$. In some cases, *e.g.* $k = 4, 16, 32$, our scheme even offers fewer multiplications and/or squarings than the scheme in [6].

Only two precomputation schemes with just one inversion have been proposed in the literature [7][12], where the Longa-Miri scheme [12] offers a better

[3] In the case of $k = 4$, we can firstly compute $[3]P$, then $2 \cdot [3]P$ and $[6]P \pm P$ with $3\mathsf{I} + 8\mathsf{S} + 13\mathsf{M}$.

[4] In the case of $k = 32$, if we set $u_0 = 3, n_0 = 8$, the precomputation stage costs $4\mathsf{I} + 137\mathsf{M} + 53\mathsf{S}$.

performance. In this section, we make a performance comparison among straightforward algorithm in affine coordinates, the algorithm in [12], and our scheme. We also analyze and suggest which scheme obtains the highest performance with a particular range of I/M and a particular width of the window. The analysis is showed in Table 4.

Table 4. Compare the performance of precomputation schemes. Here, the Longa-Miri's algorithm uses $(4k-3)$ registers, while the straightforward algorithm and our algorithm requires $2(k-1)$ registers.

k	Longa-Miri [12] (1)	Straightforward Algorithm (2)	Our algorithm (3)	Highest performance
4	$1\mathsf{I}+27\mathsf{M}+12\mathsf{S}$	$4\mathsf{I}+8\mathsf{M}+5\mathsf{S}$	$3\mathsf{I}+13\mathsf{M}+8\mathsf{S}$	(2) if $\mathsf{I}<7.4\mathsf{M}$ (3) if $7.4\mathsf{M}<\mathsf{I}<8.6\mathsf{M}$ (1) if $\mathsf{I}>8.6\mathsf{M}$
8	$1\mathsf{I}+63\mathsf{M}+20\mathsf{S}$	$8\mathsf{I}+16\mathsf{M}+9\mathsf{S}$	$3\mathsf{I}+38\mathsf{M}+18\mathsf{S}$	(2) if $\mathsf{I}<5.8\mathsf{M}$ (3) if $5.8\mathsf{M}<\mathsf{I}<13.3\mathsf{M}$ (1) if $\mathsf{I}>13.3\mathsf{M}$
16	$1\mathsf{I}+135\mathsf{M}+36\mathsf{S}$	$16\mathsf{I}+32\mathsf{M}+17\mathsf{S}$	$4\mathsf{I}+68\mathsf{M}+28\mathsf{S}$	(2) if $\mathsf{I}<3.8\mathsf{M}$ (3) if $3.8\mathsf{M}<\mathsf{I}<24.5\mathsf{M}$ (1) if $\mathsf{I}>24.5\mathsf{M}$
32	$1\mathsf{I}+279\mathsf{M}+68\mathsf{S}$	$32\mathsf{I}+64\mathsf{M}+33\mathsf{S}$	$5\mathsf{I}+132\mathsf{M}+46\mathsf{S}$	(2) if $\mathsf{I}<4.6\mathsf{M}$ (3) if $4.6\mathsf{M}<\mathsf{I}<41.1\mathsf{M}$ (1) if $\mathsf{I}>41.1\mathsf{M}$

As showed in Table 4, the answer to the question "*which scheme is faster?*", this strongly depends on the I/M-ratios and the value of k. When the I/M-ratios are large (*e.g.* $\mathsf{I}/\mathsf{M} \geq 25$), the Longa-Miri scheme offers the best performance in the cases of $k \leq 16$. However, in many applications (e.g. pairing-based cryptography), when the I/M-ratios are around 10 or higher (see benchmarks in [1]), our scheme is the best choice for $k \geq 8$. Furthermore, our algorithm only requires $2(k-1)$ registers while the Longa-Miri algorithm used in this comparison requires $4k-3$ registers.

6 Conclusion

In this paper, we described the efficient precomputation scheme on Weierstrass elliptic curves in affine coordinates. Our scheme not only requires fewer inversions, but also in some cases fewer multiplications and squarings in comparison to the schemes in the same approach. We analyzed in detail the computational complexity of our schemes, compared to the existing state-of-the-art schemes, and pointed out which scheme offers the highest performance in particular situations.

Acknowledgments. The authors thank Prof. James Davenport for his detailed and helpful comments on the manuscript, and the anonymous referees for their valuable feedback.

References

1. Acar, T., Lauter, K., Naehrig, M., Shumow, D.: Affine Pairings on ARM. Cryptology ePrint Archive, Report 2011/243 (2011), `http://eprint.iacr.org/`
2. Brown, M., Hankerson, D., López, J., Menezes, A.: Software Implementation of the NIST Elliptic Curves Over Prime Fields. In: Naccache, D. (ed.) CT-RSA 2001. LNCS, vol. 2020, pp. 250–265. Springer, Heidelberg (2001)
3. Ciet, M., Joye, M., Lauter, K., Montgomery, P.: Trading inversions for multiplications in elliptic curve cryptography. Designs, Codes and Cryptography 39, 189–206 (2006), doi:10.1007/s10623-005-3299-y
4. Cohen, H.: A course in computational algebraic number theory. Springer-Verlag New York, Inc., New York (1993)
5. Cohen, H.: Analysis of the sliding window powering algorithm. Journal of Cryptology 18, 63–76 (2005), doi:10.1007/s00145-004-0218-8
6. Cohen, H., Miyaji, A., Ono, T.: Efficient Elliptic Curve Exponentiation using Mixed Coordinates. In: Ohta, K., Pei, D. (eds.) ASIACRYPT 1998. LNCS, vol. 1514, pp. 51–65. Springer, Heidelberg (1998)
7. Dahmen, E., Okeya, K., Schepers, D.: Affine Precomputation with Sole Inversion in Elliptic Curve Cryptography. In: Pieprzyk, J., Ghodosi, H., Dawson, E. (eds.) ACISP 2007. LNCS, vol. 4586, pp. 245–258. Springer, Heidelberg (2007)
8. Hankerson, D., Menezes, A.J., Vanstone, S.: Guide to Elliptic Curve Cryptography. Springer-Verlag New York, Inc., Secaucus (2003)
9. Koblitz, N.: Elliptic curve cryptosystems. Mathematics of Computation 48(177), 203–209 (1987)
10. Le, D.-P.: Fast Quadrupling of a Point in Elliptic Curve Cryptography. Cryptology ePrint Archive, Report 2011/039 (2011), `http://eprint.iacr.org/`
11. Longa, P., Gebotys, C.: Novel Precomputation Schemes for Elliptic Curve Cryptosystems. In: Abdalla, M., Pointcheval, D., Fouque, P.-A., Vergnaud, D. (eds.) ACNS 2009. LNCS, vol. 5536, pp. 71–88. Springer, Heidelberg (2009)
12. Longa, P., Miri, A.: New Composite Operations and Precomputation Scheme for Elliptic Curve Cryptosystems over Prime Fields. In: Cramer, R. (ed.) PKC 2008. LNCS, vol. 4939, pp. 229–247. Springer, Heidelberg (2008)
13. Meloni, N.: New Point Addition Formulae for ECC Applications. In: Carlet, C., Sunar, B. (eds.) WAIFI 2007. LNCS, vol. 4547, pp. 189–201. Springer, Heidelberg (2007)
14. Miller, V.S.: Use of Elliptic Curves in Cryptography. In: Williams, H.C. (ed.) CRYPTO 1985. LNCS, vol. 218, pp. 417–426. Springer, Heidelberg (1986)
15. Okeya, K., Sakurai, K.: Fast Multi-scalar Multiplication Methods on Elliptic Curves with Precomputation Strategy using Montgomery Trick. In: Kaliski Jr., B.S., Koç, Ç.K., Paar, C. (eds.) CHES 2002. LNCS, vol. 2523, pp. 564–578. Springer, Heidelberg (2003)

16. Okeya, K., Takagi, T., Vuillaume, C.: Efficient Representations on Koblitz Curves with Resistance to Side Channel Attacks. In: Boyd, C., González Nieto, J.M. (eds.) ACISP 2005. LNCS, vol. 3574, pp. 218–229. Springer, Heidelberg (2005), doi:10.1007/11506157_19
17. Scott, M.: New record breaking implementations of ECC on quadratic extensions using endomorphisms. In: Invited talk at the 12th Workshop on Elliptic Curve Cryptography, ECC 2008 (2008)
18. Seysen, M.: Using an RSA Accelerator for Modular Inversion. In: Rao, J.R., Sunar, B. (eds.) CHES 2005. LNCS, vol. 3659, pp. 226–236. Springer, Heidelberg (2005)